

IntelliFlow — Codebase Guide

This document explains what every part of the IntelliFlow codebase does, in plain language. It is not a coding tutorial — it is a map of the project so you always know where to find things and what each file is responsible for.

Read this alongside the project proposal. This document answers 'where is this feature in the code?' while the proposal answers 'what does this feature do?'

1. Project Folder Structure

Every file in IntelliFlow lives in one of these folders. The structure is designed so that each engine is completely independent — you can work on AutoML without touching the agent code.

```
intelliflow/          ← root project folder
|
├─ core/              ← shared code used by all engines
│  └─ data/           ← everything to do with loading and storing data
│  └─ config/         ← settings and configuration files
│  └─ utils/          ← helper functions (logging, file handling)
|
├─ engines/           ← the three service engines
│  └─ automl/          ← Engine 1: AutoML pipeline
│  └─ analytics/      ← Engine 2: EDA and charts
│  └─ agents/         ← Engine 3: AI agent crew
|
├─ api/               ← FastAPI web application
│  └─ main.py          ← app entry point
│  └─ routers/        ← one router file per engine
|
├─ ui/                ← Streamlit dashboard
│  └─ app.py           ← main dashboard file
|
├─ docker-compose.yml ← starts all services
├─ config.yaml         ← all settings in one place
└─ requirements.txt    ← all Python packages
```

2. Core — Shared Data Layer

2.1 What this folder does

The core/data folder contains the code that reads files, cleans them, and makes them available to all three engines. This is the most important part of the project — if this works well, the engines are easy to build.

2.2 Key files

core/data/dataset_manager.py — The main class that handles all data. When a user uploads a file, this class reads it, checks for errors, figures out what type each column is, and stores everything ready for the engines.

It does the following things:

- Reads CSV and Excel files using Pandas
- Detects whether each column is a number, a category, or a date
- Counts missing values and flags columns with too many
- Gives the dataset a unique ID so the engines can find it later

core/data/connectors.py — Alternative ways to load data beyond file uploads — for example, reading from a database or a URL. Not required for the minimum viable version, but planned.

core/data/validator.py — Checks that incoming data makes sense — for example, it checks that the target column the user selected actually exists in the dataset.

core/config/settings.py — Reads the config.yaml file and makes settings available throughout the project. Settings include things like how many Optuna trials to run, which MLflow server to use, and which AI model to call.

3. Engine 1 — AutoML

3.1 What this folder does

The engines/automl folder contains every step of the automated machine learning pipeline, from data cleaning to model deployment. Each step is a separate file so you can update one step without affecting the others.

3.2 Key files and what they do

engines/automl/pipeline.py — The main controller for Engine 1. It calls the other files in the right order: first preprocess, then select features, then run HPO, then track results, then deploy. Think of it as the manager.

engines/automl/preprocessor.py — Handles all data cleaning automatically.

What it does step by step:

1. Missing values — fills in gaps with the column average (for numbers) or the most common value (for categories).
2. Encoding — converts text categories (like 'Male'/'Female') into numbers that the model can understand.
3. Scaling — adjusts number columns so they are all on the same scale (e.g., 0 to 1). This helps many models work better.

All of this happens automatically based on the data type of each column.

`engines/automl/hpo.py` — Runs the hyperparameter optimization using Optuna.

What it does:

4. Creates an Optuna study — this is the container for all optimization trials.
5. For each trial, Optuna suggests a combination of settings for one of the three model types.
6. The model is trained with those settings using cross-validation (splitting the data into 5 parts and testing on each part in turn).
7. The cross-validation score is returned to Optuna.
8. Optuna learns from the score and suggests better settings for the next trial.
9. This repeats for however many trials are configured (default: 50).

`engines/automl/tracker.py` — Connects to MLflow and logs everything about each trial.

What gets logged per trial:

- All hyperparameter settings (e.g., `n_estimators=200`, `learning_rate=0.05`)
- The cross-validation score (e.g., `AUC = 0.91`)
- The trained model file (so you can download and use it later)
- Charts like the confusion matrix and feature importance

`engines/automl/registry.py` — Takes the best-performing run from MLflow and registers it as the production model.

After registration, the model can be loaded by any part of the application using its name and version — there is no need to remember which file it was saved to.

`api/routers/automl.py` — The FastAPI endpoints for Engine 1.

Endpoints:

- `POST /automl/run` — starts a new AutoML job for a dataset
- `GET /automl/runs` — lists all past runs for a dataset
- `POST /automl/predict` — sends a row of data and gets a prediction back

4. Engine 2 — Analytics & EDA

4.1 What this folder does

The `engines/analytics` folder handles all data exploration, statistical analysis, and chart generation. It takes a dataset and produces a rich, interactive report without the user writing any code.

4.2 Key files

`engines/analytics/profiler.py` — The auto-profiler. Loops through every column and computes a set of statistics.

For every column it computes:

- Data type (number, category, date, text)
- Percentage of missing values
- Number of unique values
- For numbers: average, minimum, maximum, standard deviation, skewness
- For categories: the top 5 most common values and their counts

The profiler output is a Python dictionary that is used by both the visualizer and the API.

`engines/analytics/visualizer.py` — Creates all the Plotly charts based on the profiler output.

Charts it creates:

- Histogram — for every numeric column, shows the distribution of values
- Box plot — shows median, spread, and outliers for numeric columns
- Correlation heatmap — colour-coded grid showing how strongly columns relate to each other
- Scatter plot — shows the relationship between any two numeric columns
- Bar chart — shows category counts for text/category columns

`engines/analytics/patterns.py` — The advanced analytics patterns inspired by FAANG company practices.

What it includes:

- Funnel analysis — given a list of columns that represent steps in a process, calculates how many records complete each step and the drop-off between steps.
- Cohort retention — groups records by a date column into weekly or monthly cohorts, then calculates what percentage of each cohort is still active in later weeks.
- Event timeline — counts how many events happen per time window and plots them as a time series.

`engines/analytics/anomaly.py` — Detects unusual data points using two methods.

Method 1 — Z-score: Calculates how many standard deviations away from the average each value is. Values more than 3 standard deviations away are flagged as anomalies.

Method 2 — IQR (Interquartile Range): Flags values that are far above or below the middle 50% of values. This method works better for data that is not normally distributed.

5. Engine 3 — AI Agent

5.1 What this folder does

The engines/agents folder contains the multi-agent AI system. It uses the CrewAI framework to create a team of specialized agents that work together to answer the user's question.

5.2 Key files

`engines/agents/crew.py` — The main controller for the agent engine. Creates the crew of agents, assigns tasks, and runs the workflow when a user submits a question.

How it works:

10. The user's question and the dataset are passed in.
11. CrewAI creates the agent crew (Planner, Analyst, ML Agent, Visualizer, Synthesizer).
12. The crew runs — each agent completes its task and passes results to the next.
13. The final answer is returned to the API and displayed in the dashboard.

`engines/agents/roles.py` — Defines each agent: their name, goal, and backstory. CrewAI uses the backstory to give each agent a personality and area of expertise when talking to the AI model.

`engines/agents/tools.py` — The functions that agents can use during their work.

Available tools:

- `pandas_tool` — runs Pandas operations on the dataset (calculate mean, filter rows, group by column, etc.)
- `plotly_tool` — creates a Plotly chart and returns it as a JSON object
- `automl_tool` — triggers Engine 1 on the current dataset and returns the best model's results
- `search_tool` — does a web search for context or domain knowledge (uses DuckDuckGo, which is free)

`engines/agents/memory.py` — Stores previous questions and answers so the agent crew has context for follow-up questions.

Uses ChromaDB — a free, local vector database. When the user asks a question, the system searches previous answers to see if anything is relevant before calling the AI model. This reduces unnecessary API calls and makes responses faster.

5.3 The AI model connection

The agents are connected to the AI model through a single configuration in the crew setup. Switching between Groq and Ollama requires changing one line:

```
# Option A - Groq (free API, requires free account at groq.com)
from langchain_groq import ChatGroq
llm = ChatGroq(model='llama3-70b-8192') # free tier

# Option B - Ollama (completely local, no internet needed)
from langchain_ollama import ChatOllama
llm = ChatOllama(model='llama3') # run: ollama pull llama3
```

6. API — FastAPI Gateway

6.1 What this folder does

The api folder contains the FastAPI web application. It is the middleman between the Streamlit dashboard and the three engines. Every request from the dashboard goes through here.

6.2 Key files

api/main.py — The entry point for the API. Creates the FastAPI application, registers all routers, and sets up CORS (so the Streamlit dashboard can talk to it).

api/routers/automl.py — Handles all AutoML requests (/automl/run, /automl/predict, /automl/runs).

api/routers/analytics.py — Handles all analytics requests (/analytics/profile, /analytics/charts, /analytics/anomalies).

api/routers/agent.py — Handles all agent requests (/agent/query, /agent/history).

api/auth.py — Simple username/password login that returns a token. The token is required for all API calls. This prevents unauthorized access to the platform.

6.3 How to see all endpoints

FastAPI automatically generates interactive documentation. Once the server is running, open a browser and go to:

```
http://localhost:8000/docs
```

This shows every endpoint, what inputs it expects, and what it returns. You can test any endpoint directly from this page — no Postman or curl needed.

7. Dashboard — Streamlit UI

ui/app.py — The entire Streamlit dashboard in one file. Streamlit is a Python library that builds web UIs without needing to write HTML or JavaScript.

The dashboard has four tabs:

- Upload tab — file uploader, shows a preview of the data and the column summary from the profiler
- Analytics tab — runs Engine 2 and displays all the charts and the profiling report
- AutoML tab — lets the user pick a target column, start a training run, and see results; links to the MLflow dashboard
- Agent tab — a chat interface where the user types questions and sees the agent's answers

8. Deployment — Docker Compose

8.1 Services

Service name	What it is	Port
api	The FastAPI application — the core of IntelliFlow	8000
mlflow	The MLflow tracking server — experiment history UI	5000
streamlit	The Streamlit dashboard — what users see	8501
redis	A fast in-memory cache — speeds up repeated requests	6379
chromadb	The vector database for agent memory	8001

8.2 How to run the project

14. Install Docker Desktop (free, works on Windows, Mac, Linux).

15. Clone or download the IntelliFlow project folder.

16. Create a file called `.env` in the root folder and add your Groq API key:

```
GROQ_API_KEY=your_free_key_here
```

17. Open a terminal in the project folder and run:

```
docker compose up --build
```

18. Open your browser:

```
Dashboard: http://localhost:8501
```

```
API docs: http://localhost:8000/docs
```

```
MLflow: http://localhost:5000
```

8.3 For Ollama (no API key at all)

19. Install Ollama from ollama.com (free).

20. Run in terminal: `ollama pull llama3`

21. Change one line in the `crew.py` file (see Section 5.3 above).

22. Run `docker compose up` — no `.env` file or API key needed.

9. Configuration — `config.yaml`

All settings live in one file so you never have to hunt through code to change something. Key settings:

```
automl:
  n_trials: 50          # how many Optuna trials to run
  cv_folds: 5           # cross-validation folds
  metric: roc_auc       # what to optimize for (classification)
  models: [rf, xgb, lgbm] # which model families to compare
```

```
analytics:
  anomaly_threshold: 3 # Z-score threshold for flagging outliers
  max_charts: 20      # maximum charts to generate per dataset

agents:
  llm_provider: groq   # or 'ollama' for local
  model: llama3-70b-8192
  max_memory_results: 5 # how many past answers to retrieve as context
```

10. Testing

tests/test_data.py — Tests for the data layer — checks that files are parsed correctly, that missing values are handled, and that the column type detection works.

tests/test_automl.py — Tests for the AutoML engine — checks that preprocessing produces the right output, that Optuna runs without errors, and that MLflow logs are created.

tests/test_analytics.py — Tests for the analytics engine — checks that the profiler output has all required fields and that charts are generated for each chart type.

tests/test_agents.py — Tests for the agent engine — checks that the crew runs without errors on a small sample dataset.

Run all tests with one command:

```
pytest tests/ -v
```

11. Requirements — Python Packages

Package	Version	Used for
fastapi	0.110+	API framework
uvicorn	latest	Runs the FastAPI server
pandas	2.x	Data handling
numpy	1.x	Numerical operations
scipy	1.x	Statistics and anomaly detection
scikit-learn	1.x	Data preprocessing and Random Forest
xgboost	2.x	XGBoost model
lightgbm	4.x	LightGBM model
optuna	3.x	Hyperparameter optimization
mlflow	2.x	Experiment tracking
plotly	5.x	Interactive charts
crewai	0.x	Multi-agent framework

Package	Version	Used for
langchain-groq	latest	Groq API connection for agents
langchain-ollama	latest	Ollama local model connection
chromadb	0.x	Vector database for agent memory
sentence-transformers	latest	Turns text into vectors for ChromaDB
streamlit	1.35+	Web dashboard
pydantic	2.x	Data validation
python-jose	latest	JWT authentication tokens
redis	latest	Python client for Redis cache
pytest	latest	Testing framework